

# **CE-408 Cloud Computing**



## **Project Report**

### **Team Members:**

**(1):** Ijlal Ahmad (2022229)

**(2):** Aqib Muhammad (2022103)

### **Registration Number:**

**(1):** 2022229

**(2):** 2022103

**Submitted to:** Ms. Safia Baloch

**Semester:** 08

**Faculty of Computer Science and Engineering**

**GIK Institute of Engineering Sciences and Technology**

# Cloud-Based Log Analytics System

## Abstract

In modern distributed environments, the massive volume of log data generated by applications makes manual monitoring and incident detection nearly impossible, often leading to delayed response times and system downtime. This project presents the design and implementation of a Cloud-Based Log Analytics System built on Amazon Web Services (AWS) to automate real-time anomaly detection. Using a simulated application on an EC2 instance, logs are streamed via the CloudWatch Agent to a serverless pipeline. An AWS Lambda function processes these logs, triggering instant alerts through the Simple Notification Service (SNS) when errors are detected, while simultaneously archiving metadata in DynamoDB and raw logs in Amazon S3. The resulting system demonstrates an event-driven architecture that successfully reduces incident detection latency and provides a scalable and cost-effective solution for historical log auditing and system health monitoring.

## 1. Introduction

In the modern software landscape applications generate massive volumes of log data through continuous system activity and complex event streams. Relying on manual monitoring is not only inefficient but creates a dangerous operational blind spot as systems scale. Without an automated analysis pipeline, critical failures often remain invisible until they impact the end-user leading to costly downtime. This project bridges that gap by implementing serverless architecture that transforms raw log into Structured Event Records, enabling real-time anomaly detection and proactive incident response.

## 2. Objectives

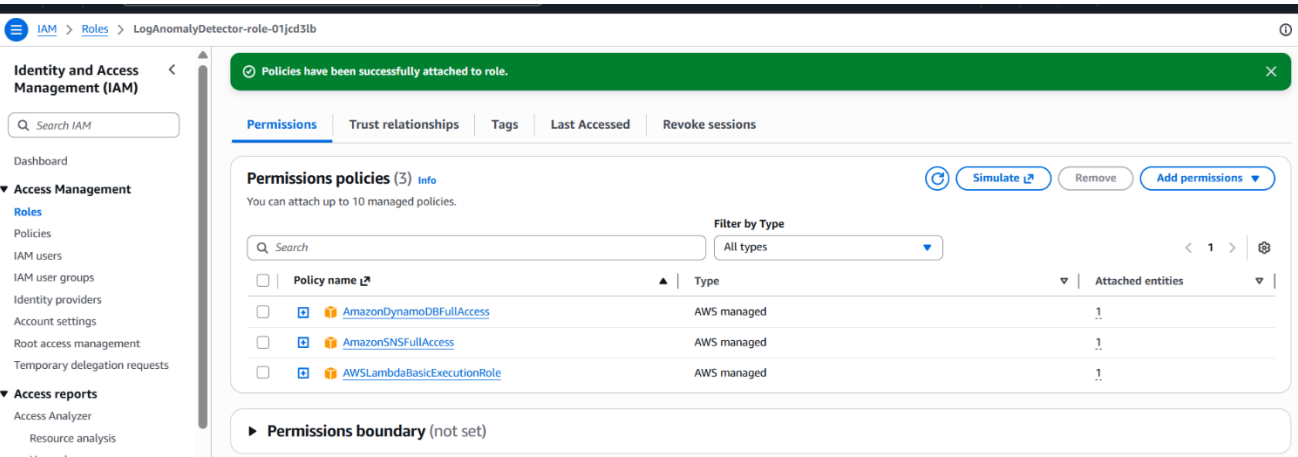
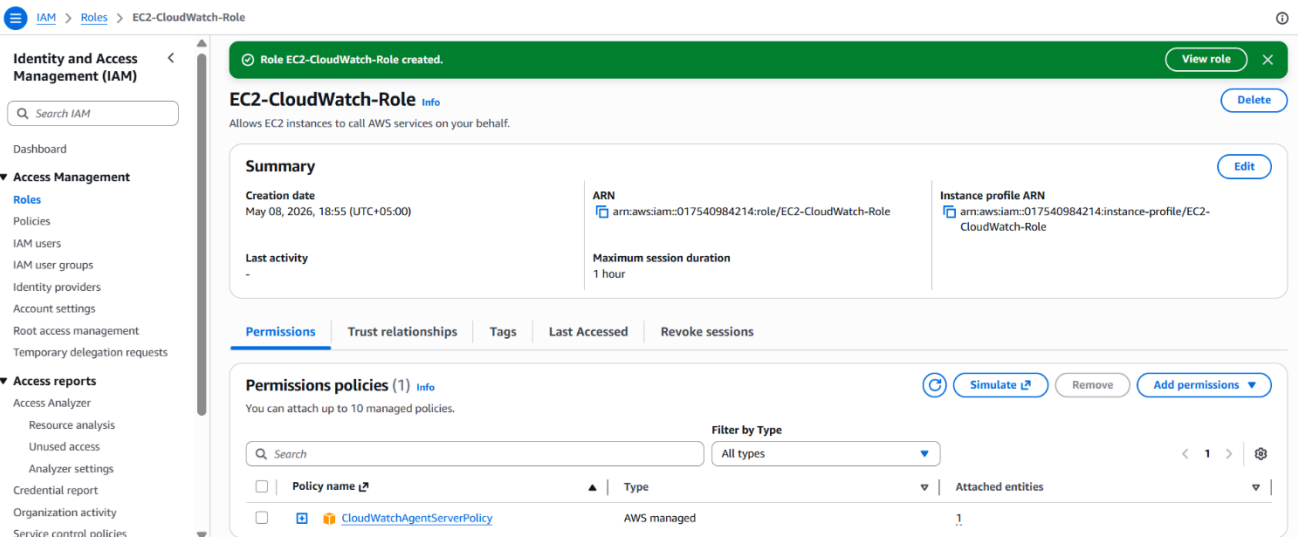
- **Automated Collection:** To establish a consistent stream of application logs from EC2 instances to the cloud without manual file transfers.
- **Real-Time Processing:** To implement an event-driven pipeline that analyzes logs as they arrive to minimize detection latency.
- **Autonomous Detection:** To programmatically identify critical errors and performance bottlenecks using serverless architecture.
- **Instant Notification:** To trigger immediate alerts via SNS to ensure the development team is notified of failures as they happen.
- **Error Registry:** To maintain Structured Incident Records in DynamoDB for rapid searching and historical trend analysis.
- **Durable Storage:** To automate the archival of raw logs in S3 for system traceability and root-cause analysis.

## 3. Methodology and System Architecture

The system follows a serverless, event-driven pipeline where each service is decoupled to ensure scalability and reliability.

### 3.1 Access Management and Roles

An **EC2-CloudWatch-Role** was created to grant the EC2 instance permission to write logs to CloudWatch. Also, a separate execution role i.e. **LogAnomalyDetector-role-01jcd3lb** was created for the Lambda function with **AmazonSNSFullAccess** and **AmazonDynamoDBFullAccess**.



**Identity and Access Management (IAM)**

Dashboard

▼ Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer

**Roles (2/8)** [Info](#)

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

| <input type="checkbox"/>            | Role name                                         | Trusted entities                                  | Last activity  |
|-------------------------------------|---------------------------------------------------|---------------------------------------------------|----------------|
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForAPIGateway</a>       | AWS Service: ops.apigateway (Service-Linker)      | -              |
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForNATGateway</a>       | AWS Service: ec2-nat-gateway (Service-Linker)     | 59 days ago    |
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForOrganizations</a>    | AWS Service: organizations (Service-Linker)       | -              |
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForResourceExplorer</a> | AWS Service: resource-explorer-2 (Service-Linker) | 1 hour ago     |
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForSupport</a>          | AWS Service: support (Service-Linker)             | -              |
| <input type="checkbox"/>            | <a href="#">AWSServiceRoleForTrustedAdvisor</a>   | AWS Service: trustedadvisor (Service-Linker)      | -              |
| <input checked="" type="checkbox"/> | <a href="#">EC2-CloudWatch-Role</a>               | AWS Service: ec2                                  | 13 minutes ago |
| <input checked="" type="checkbox"/> | <a href="#">LogAnomalyDetector-role-01jcd3lb</a>  | AWS Service: lambda                               | 27 minutes ago |

## 3.2 Environment and Log Generation

An **Ubuntu 22.04 LTS** instance was launched on **EC2 (t3. micro)**. A Python script (**log\_generator.py**) was deployed to simulate a live application generating INFO, WARNING, and ERROR logs.

**EC2 > Instances > Launch an instance**

Recents Quick Start

Amazon Linux macOS **Ubuntu** Windows Red Hat SUSE Linux Debian

**Amazon Machine Image (AMI)**

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type  
ami-091138d0f0d41ff90 (64-bit (x86)) / ami-07ad186bc37b8dac4 (64-bit (Arm))  
Virtualization: hvm ENA enabled: true Root device type: ebs

**Description**

Ubuntu Server 26.04 LTS (HVM), EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).  
Canonical, Ubuntu, 26.04, amd64 resolute image

**Architecture** 64-bit (x86) **AMI ID** ami-091138d0f0d41ff90 **Publish Date** 2026-04-21 **Username** ubuntu **Verified provider**

**▼ Instance type** [Info](#) [Get advice](#)

Instance type **t3.micro**

**▼ Summary**

**Number of instances** 1

**Software Image (AMI)**  
Canonical, Ubuntu, 26.04, amd64... [read more](#)  
ami-091138d0f0d41ff90

**Virtual server type (instance type)**  
t3.micro

**Firewall (security group)**  
New security group

**Storage (volumes)**  
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

**EC2 > Instances > i-0784ad29ea3cbc6ee**

**EC2**

Dashboard  
AWS Global View  
Events

▼ **Instances**

- Instances
- Instance Types
- Launch Templates
- Spot Requests
- Savings Plans
- Reserved Instances
- Dedicated Hosts
- Capacity Reservations
- Capacity Manager [New](#)

▼ **Images**

- AMIs
- AMI Catalog

▼ **Elastic Block Store**

- Volumes
- Snapshots
- Lifecycle Manager

**Instance summary for i-0784ad29ea3cbc6ee (LogAnalyticsApp)** [Info](#)

Updated less than a minute ago

**Instance ID**  
[i-0784ad29ea3cbc6ee](#)

**IPv6 address**  
-

**Hostname type**  
IP name: ip-172-31-45-204.ec2.internal

**Answer private resource DNS name**  
IPv4 (A)

**Auto-assigned IP address**  
[3.80.255.240](#) [Public IP]

**IAM role**  
[EC2-CloudWatch-Role](#)

**IMDSv2**  
Required

**Operator**  
-

**Public IPv4 address**  
[3.80.255.240](#) [open address](#)

**Instance state**  
Running

**Private IP DNS name (IPv4 only)**  
[ip-172-31-45-204.ec2.internal](#)

**Instance type**  
t3.micro

**VPC ID**  
[vpc-07a3bc75a751c4996](#)

**Subnet ID**  
[subnet-0d9e443a8a0bfd3c6](#)

**Instance ARN**  
[arn:aws:ec2:us-east-1:017540984214:instance/i-0784ad29ea3cbc6ee](#)

**Private IPv4 addresses**  
[172.31.45.204](#)

**Public DNS**  
[ec2-3-80-255-240.compute-1.amazonaws.com](#) [open address](#)

**Elastic IP addresses**  
-

**AWS Compute Optimizer finding**  
[Opt-in to AWS Compute Optimizer for recommendations.](#) [Learn more](#)

**Auto Scaling Group name**  
-

**Managed**  
false

### log\_generator.py

```
import logging
import time
import random
import os

# — Setup
LOG_FILE = "/var/log/myapp/app.log"
os.makedirs("/var/log/myapp", exist_ok=True)

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.DEBUG,
    format="%(asctime)s %(levelname)s %(message)s",
    datefmt="%Y-%m-%dT%H:%M:%S"
)

# — Events
INFO_EVENTS = [
    "User logged in successfully",
    "File uploaded successfully",
    "API request completed",
    "User profile updated",
    "Payment processed successfully",
    "Session started",
    "Data fetched from database",
]

WARNING_EVENTS = [
    "Database response slow (2.3s)",
    "Memory usage above 70%",
    "API response time high",
    "Disk space below 20%",
    "Retry attempt on failed request",
]

ERROR_EVENTS = [
    "Failed to connect to database",
    "Payment gateway timeout",
    "Unhandled exception in API handler",
    "File not found: config.json",
    "Authentication service unreachable",
    "OutOfMemory: Java heap space",
    "CRITICAL: Segmentation fault in worker",
]

# — Mode control
# Change this to "error" during your demo to trigger alerts
```

```

MODE = "normal"

def generate_log():
    if MODE == "normal":
        weights = [75, 20, 5] # mostly INFO, rare errors
    else:
        weights = [10, 10, 80] # flood of errors — triggers alerts

    level = random.choices(["INFO", "WARNING", "ERROR"], weights=weights)[0]

    if level == "INFO":
        logging.info(random.choice(INFO_EVENTS))
    elif level == "WARNING":
        logging.warning(random.choice(WARNING_EVENTS))
    else:
        logging.error(random.choice(ERROR_EVENTS))

# — Main loop
print("Log generator started. Writing to", LOG_FILE)
print("Press Ctrl+C to stop.")

while True:
    generate_log()
    time.sleep(2) # new log every 2 seconds

```

### 3.3 Log Streaming and Configuration

The **CloudWatch Agent** was installed on the EC2 instance. A configuration file (**cloudwatch-config.json**) mapped the local file path **/var/log/myapp/app.log** to the CloudWatch Log Group **LogAnalyticsSystem**.

```

ubuntu@ip-172-31-45-204: ~$ wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
--2026-05-08 14:48:25-- https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
Resolving s3.amazonaws.com (s3.amazonaws.com)... 52.217.196.40, 52.217.73.38, 16.15.244.203, ...
Connecting to s3.amazonaws.com (s3.amazonaws.com)|52.217.196.40|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 67828572 (65M) [application/octet-stream]
Saving to: 'amazon-cloudwatch-agent.deb'

amazon-cloudwatch-agent.deb      100%[=====>] 64.69M  98.8MB/s   in 0.7s

2026-05-08 14:48:26 (98.8 MB/s) - 'amazon-cloudwatch-agent.deb' saved [67828572/67828572]

ubuntu@ip-172-31-45-204:~$ sudo dpkg -i amazon-cloudwatch-agent.deb
Selecting previously unselected package amazon-cloudwatch-agent.
(Reading database ... 85301 files and directories currently installed.)
Preparing to unpack amazon-cloudwatch-agent.deb ...
create group cwagent, result: 0
create user cwagent, result: 0
Unpacking amazon-cloudwatch-agent (1.300067.0b1404-1) ...
Setting up amazon-cloudwatch-agent (1.300067.0b1404-1) ...
ubuntu@ip-172-31-45-204:~$

```

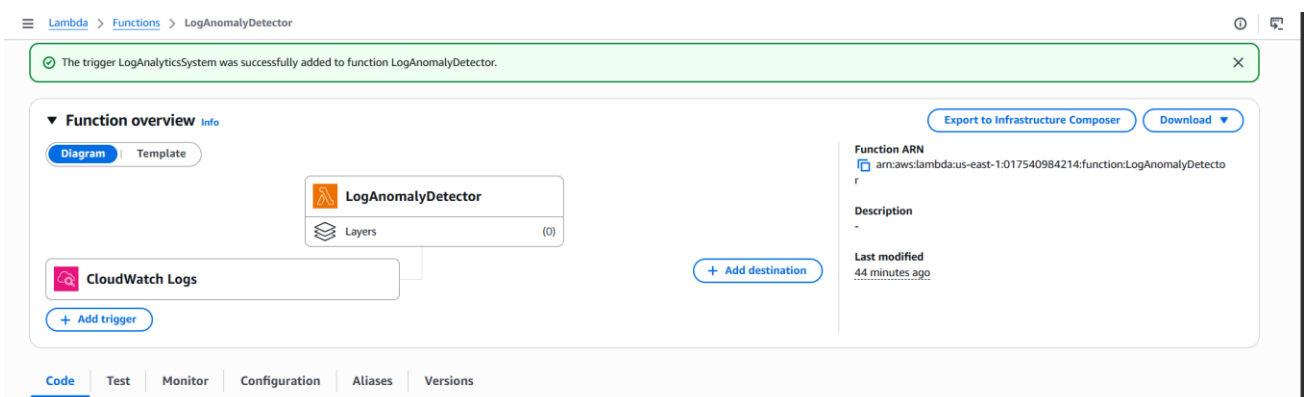
```
ubuntu@ip-172-31-45-204: ~$ sudo mkdir -p /var/log/myapp
ubuntu@ip-172-31-45-204: ~$ sudo chown ubuntu:ubuntu /var/log/myapp
ubuntu@ip-172-31-45-204: ~$ ls -ld /var/log/myapp
drwxr-xr-x 2 ubuntu ubuntu 4096 May  8 14:53 /var/log/myapp
ubuntu@ip-172-31-45-204: ~$ python3 ~/log_generator.py
Log generator started. Writing to /var/log/myapp/app.log
Press Ctrl+C to stop.
```

```
GNU nano 8.7.1 /opt/aws/amazon-cloudwatch-agent/etc/cloudwatch-config.json *
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/myapp/app.log",
            "log_group_name": "LogAnalyticsSystem",
            "log_stream_name": "EC2-AppLogs",
            "timestamp_format": "%Y-%m-%dT%H:%M:%S",
            "timezone": "UTC"
          }
        ]
      },
      "log_stream_name": "EC2-AppLogs",
      "force_flush_interval": 5
    }
  }
}
```

### 3.4 Serverless Processing (AWS Lambda)

A Python-based Lambda function was configured as a trigger for the CloudWatch Log Group. The function decompresses the incoming log batch, scans for the keyword "ERROR," and executes two concurrent actions:

1. **Alerting:** Publishes a message to an SNS Topic.
2. **Archiving:** Writes a structured record (Timestamp, Message) to DynamoDB.



### Function code

```
import json
import base64
import gzip
import boto3
from datetime import datetime

# Initialize AWS Services
sns = boto3.client('sns')
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('DetectedAnomalies')

# Replace this with your actual SNS Topic ARN from Step 12
SNS_TOPIC_ARN = "arn:aws:sns:us-east-1:017540984214:LogAlerts"

def lambda_handler(event, context):
    cw_data = event['awslogs']['data']
    compressed_payload = base64.b64decode(cw_data)
    uncompressed_payload = gzip.decompress(compressed_payload)
    payload = json.loads(uncompressed_payload)

    for log_event in payload['logEvents']:
        message = log_event['message']

        if "ERROR" in message:
            timestamp = datetime.now().isoformat()

            # 1. Send SNS Alert
            sns.publish(
                TopicArn=SNS_TOPIC_ARN,
                Subject="LOG ANALYTICS ALERT",
                Message=f"Anomaly detected:\n\n{message}"
            )

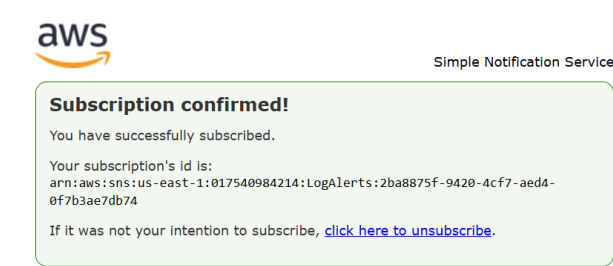
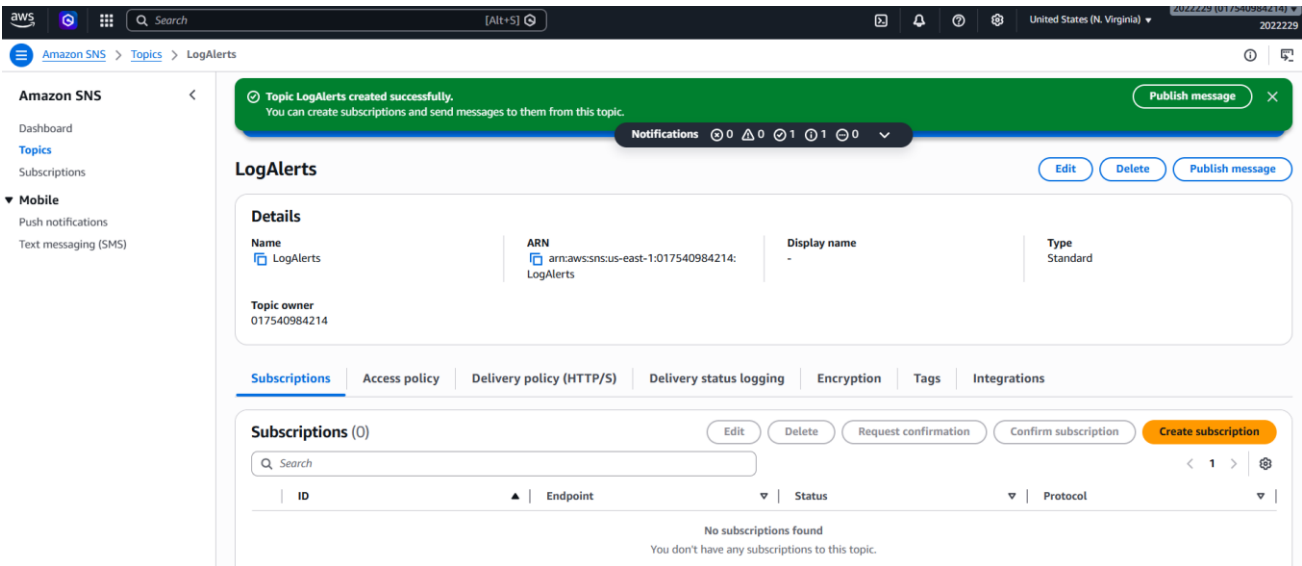
            # 2. Save to DynamoDB
            table.put_item(
                Item={
                    'Timestamp': timestamp,
                    'LogMessage': message,
                    'Status': 'Critical'
                }
            )

    return {"statusCode": 200, "body": "Logged to DynamoDB"}
```



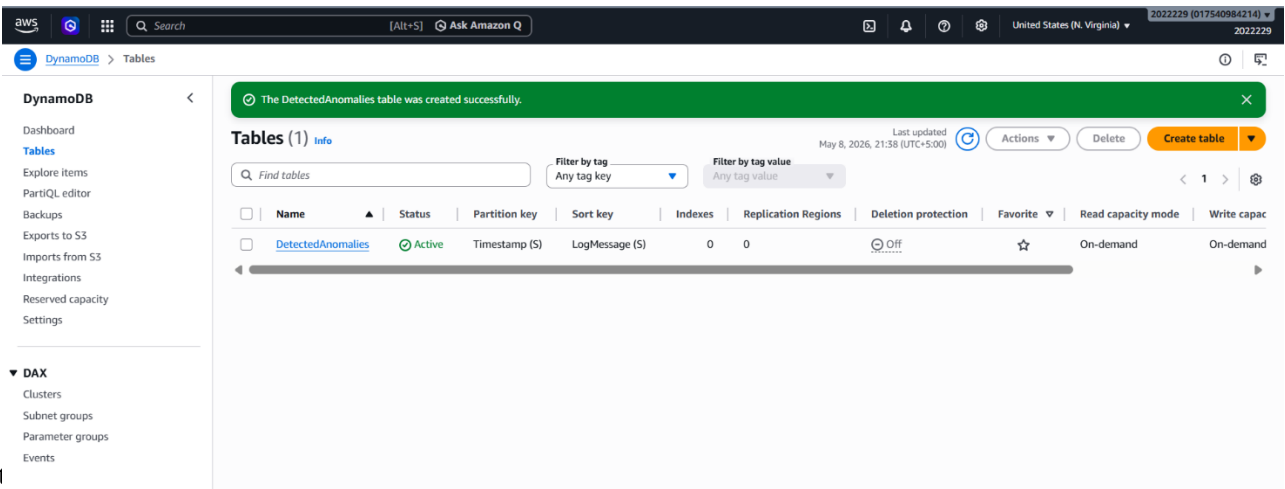
### 3.5 Notification Service (SNS)

An SNS Topic named LogAlerts was configured with an email subscription to provide instant human notification of system anomalies.



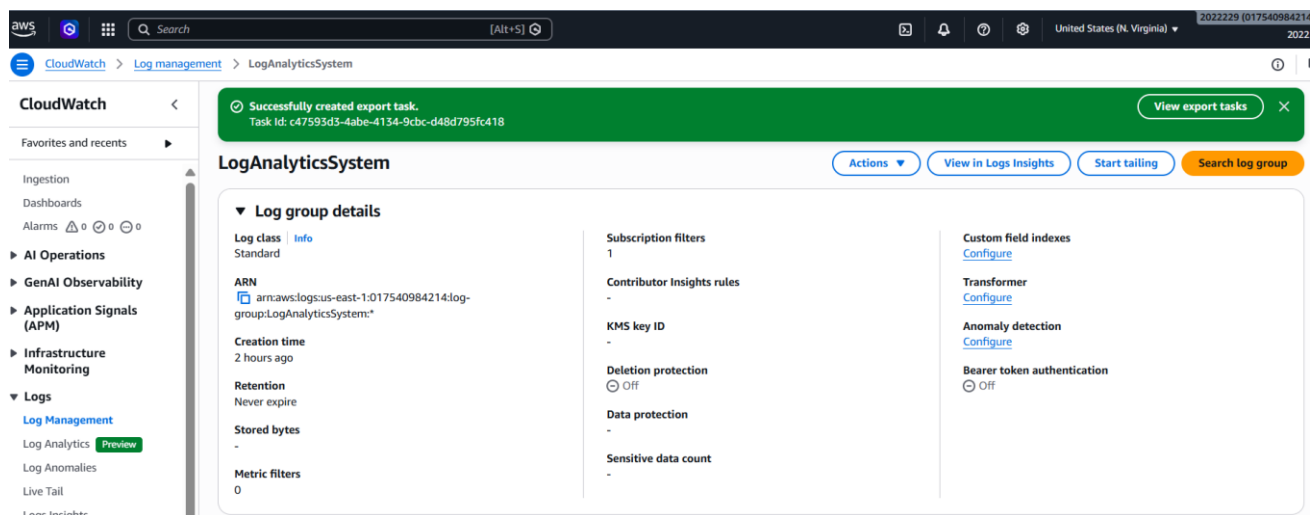
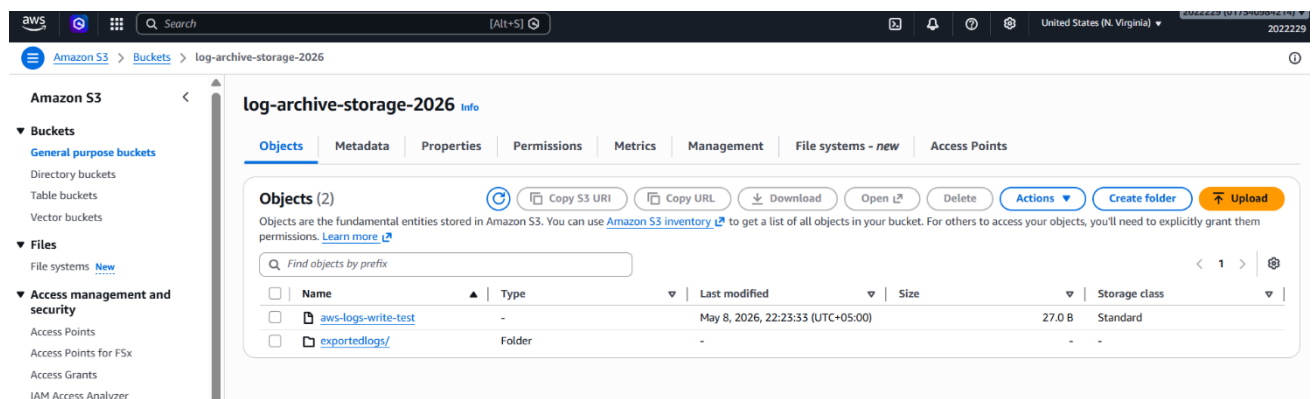
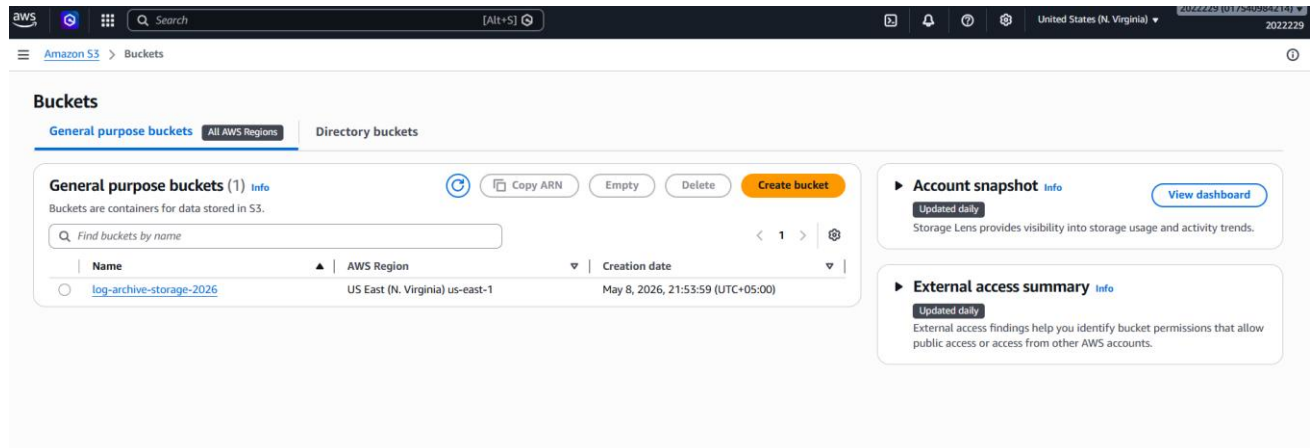
### 3.6 Database Storage (DynamoDB)

The DetectedAnomalies table serves as a permanent registry of all critical events in DynamoDB for rapid searching and historical trend analysis..



### 3.7 Long-term Archival (S3)

For system traceability and root-cause analysis, raw logs were exported to an S3 bucket i.e. **log-archive-storage-2026**. A Bucket Policy was applied to allow CloudWatch Logs access to the storage resource.



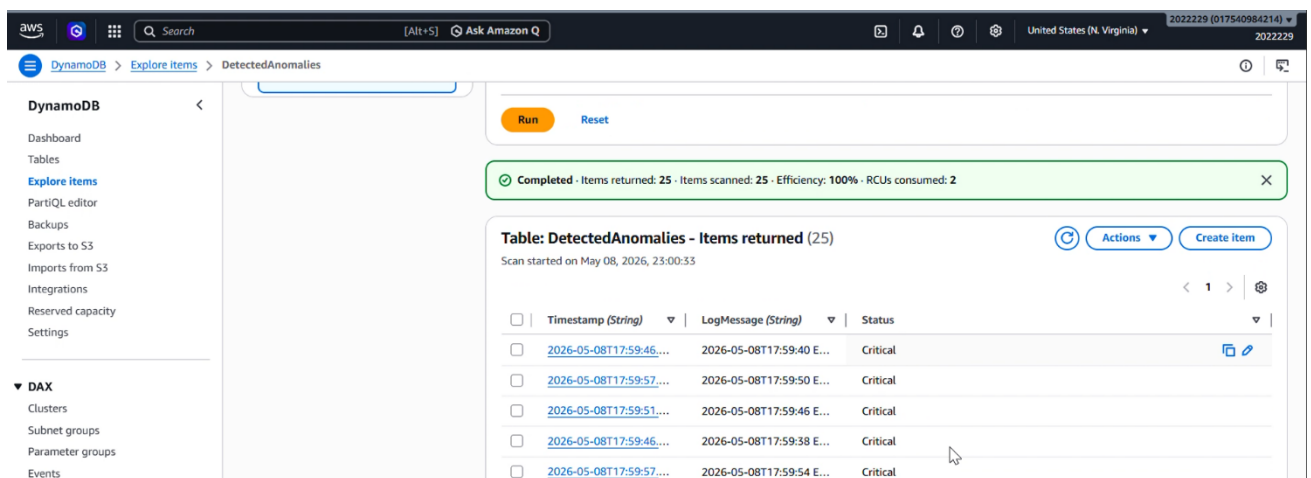
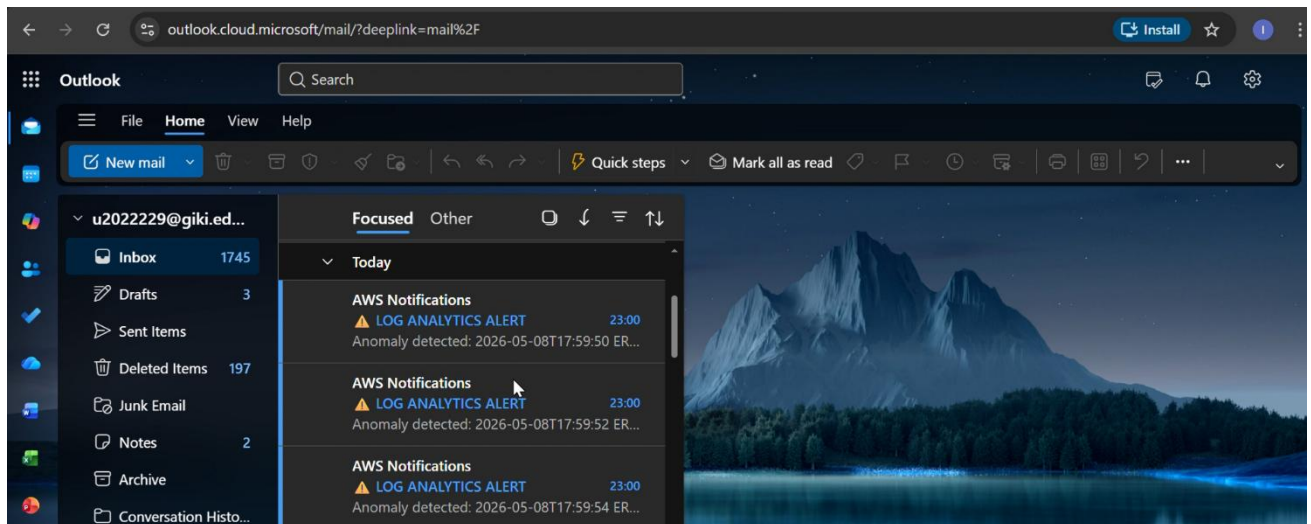
## 4. Testing & Results

The pipeline was validated by triggering a simulated system failure within the log generator.

- **Triggering and Notification:** Upon detecting a critical error, an alert was immediately dispatched via SNS, reaching the administrator's inbox with minimal latency.
- **Structured Data Capture:** A corresponding **Anomaly Record** was programmatically generated and committed to the **DynamoDB** table, ensuring high-speed access to incident data.
- **Durable Archival Execution:** The raw log stream was successfully captured and transitioned to the **Amazon S3** bucket, confirming the system's capacity for **long-term troubleshooting and root-cause analysis**.
- **Outcome:** The system successfully demonstrated error detection and reporting.

```
ubuntu@ip-172-31-45-204: ~  
ubuntu@ip-172-31-45-204:~$ python3 ~/log_generator.py &  
[1] 3795  
ubuntu@ip-172-31-45-204:~$ Log generator started. Writing to /var/log/myapp/app.log  
Press Ctrl+C to stop.
```

```
ubuntu@ip-172-31-45-204:~$ tail -f /var/log/myapp/app.log  
2026-05-08T17:28:37 ERROR File not found: config.json  
2026-05-08T17:28:39 WARNING API response time high  
2026-05-08T17:28:41 ERROR OutOfMemory: Java heap space  
2026-05-08T17:28:43 WARNING Memory usage above 70%  
2026-05-08T17:28:45 ERROR CRITICAL: Segmentation fault in worker  
2026-05-08T17:28:47 ERROR Authentication service unreachable  
2026-05-08T17:28:49 ERROR Unhandled exception in API handler  
2026-05-08T17:59:38 ERROR Authentication service unreachable  
2026-05-08T17:59:40 ERROR OutOfMemory: Java heap space  
2026-05-08T17:59:42 ERROR Failed to connect to database
```



## 5. Conclusion

This project successfully established a scalable, and cloud-native log analytics system that replaces manual monitoring with a fully automated observability framework. By integrating EC2, CloudWatch, Lambda, SNS, and DynamoDB, the event-driven pipeline identifies and alerts on critical failures with minimal propagation delay. The solution ensures long-term operational reliability through Anomaly Logging and Durable Archival in Amazon S3, facilitating rapid root-cause analysis. Ultimately, this architecture demonstrates the power of serverless computing in reducing incident response times and maintaining a structured data-driven environment for proactive troubleshooting.